

1. DEFINICIÓN DE LA TAREA Y GROUND TRUTH

Contexto: El problema de asignación de turnos médicos (*Medical Staff Scheduling Problem*, MSSP) es un desafío combinatorio NP-hard de satisfacción de restricciones (CSP) crítico en la gestión hospitalaria.

Definición de la Tarea: Dado un conjunto de personal médico $S = \{s_1, \dots, s_N\}$ (con especialidades, horas máximas semanales por contrato H_i e indisponibilidades solicitadas U_i) y la demanda semanal $D = \{d_1, \dots, d_{21}\}$ en 3 turnos de 8h (*Mañana, Tarde, Noche*), el modelo debe generar la matriz de asignación completa en formato JSON válido.

Ground Truth y Evaluador Programático: La salida se evalúa mediante un oráculo determinista en Python (`src/verifier.py`) que verifica 0 violaciones en 6 Restricciones Duras (*Hard Constraints*, HC):

- **HC1 (Descanso Obligatorio):** ≥ 16 h continuas de descanso entre turnos (un turno Noche no puede ser seguido por Mañana o Tarde al día siguiente).
- **HC2 (Turno Único Diario):** Máximo 1 turno (8h) por médico al día.
- **HC3 (Demanda de Dotación):** Cantidad exacta de personal por turno.
- **HC4 (Cobertura de Especialidad):** Cuota mínima de especialistas (ej. ≥ 1 Anestesiólogo en Noche, ≥ 1 Urgenciólogo en Día).
- **HC5 (Límite Legal Semanal):** Horas totales asignadas $\leq H_i$ (≤ 40 h).
- **HC6 (Indisponibilidad):** Cero asignaciones en franjas bloqueadas.

2. DIAGNÓSTICO DEL FALLO Y EVIDENCIA

El *prompting* directo (zero-shot) en modelos compactos (≤ 8 B) falla sistemáticamente debido a tres limitaciones estructurales:

- Decisión Voraz vs. Búsqueda Global CSP:** Al generar el horario de forma autorregresiva (lunes a domingo) sin retroceso (*backtracking*), las asignaciones iniciales agotan vorazmente las horas de especialistas, forzando violaciones inevitables hacia el final de la semana.
- Deriva de Estado y Aritmética:** El modelo no mantiene contadores exactos de horas acumuladas ($\sum h_i$), generando sobrecargas ilegales (HC5) y turnos duplicados (HC2).
- Alucinación de Esquema e Identificadores:** En salidas largas, los modelos suelen truncar claves JSON, repetir IDs en un mismo turno o asignar personal inexistente.

Evidencia Empírica de Línea Base: Evaluamos $N = 10$ intentos independientes de generación *zero-shot* (con variaciones de temperatura y semillas sobre modelos de 3B) sobre el escenario de 10 médicos y 21 turnos evaluados con el oráculo (`src/verifier.py`):

- **Tasa de Horarios Válidos (0 %):** 0/10 intentos satisficieron simultáneamente las 6 restricciones duras (criterio estricto de factibilidad CSP).
- **Promedio de 7.4 Violaciones/Intento:** Media aritmética ($\mu = 7.4$, rango $[5, 13]$) de violaciones duras por corrida, concentradas principalmente en descanso insufi-

ciente HC1 (23 %), exceso de horas semanales HC5 (41 %) e indisponibilidad HC6 (28 %).

3. MODELOS CANDIDATOS Y JUSTIFICACIÓN

Para abordar la tarea con alta eficiencia computacional y rápida iteración, seleccionamos tres modelos compactos de última generación en el rango de **3.0B a 3.8B** (dentro del límite de 8B):

Modelo	Params	Ctx	GSM8k	Justificación Arquitectural
Qwen 2.5 3B-Instruct	3.09B	128k	84.5 %	SOTA en JSON estructurado y seguimiento de restricciones lógicas.
Phi-3.5-mini-Instruct	3.82B	128k	86.0 %	Datos sintéticos de alto razonamiento; sobresale en lógica paso a paso.
Llama 3.2 3B-Instruct	3.21B	128k	77.2 %	Modelo ligero y veloz; óptimo para fine-tuning eficiente con LoRA.

Justificación de Escala Compacta: El rango de ≈ 3 B ofrece latencia sub-segundo, mínimo consumo de memoria para fine-tuning (PEFT/LoRA) y permite demostrar que marcos de razonamiento estructurado (CoT / Tool-calling / Verificador) resuelven eficazmente problemas CSP complejos.

4. FACTIBILIDAD DE EJECUCIÓN

La configuración propuesta está 100 % validada para ejecutarse en el hardware disponible:

- **Hardware Local Dedicado:** Estación de trabajo con AMD Ryzen 7 7700, GPU Nvidia RTX 5060 (8 GB VRAM) y 32 GB RAM DDR5. Soporta holgadamente inferencia completa en FP16 (≈ 6.5 GB) y 4-bit NF4 (≈ 2.4 GB), así como *fine-tuning* con PEFT/LoRA sin restricciones de cuotas.
- **Google Colab Gratuito (GPU Nvidia T4, 15 GB VRAM):** Entorno en la nube 100 % reproducible para todo el equipo; ocupa ≈ 2.4 GB en 4-bit NF4, dejando > 12 GB libres (> 80 % de margen de VRAM) para memoria de contexto KV y oráculos.

Demostrado de extremo a extremo en `Colab_Feasibility.ipynb`. La huella pico (< 3.0 GB) garantiza que el flujo corre de forma estable en el hardware accesible sin riesgo de desbordamiento de memoria (*Out-Of-Memory*).

5. PLAN SEMESTRAL Y REPOSITORIO

Estrategia de Solución Semestral: Atacaremos el fallo mediante: (1) *Chain-of-Thought* multietapa con verificación intermedia, (2) *Tool-Calling* integrando oráculos y *solvers* CSP (Z3/OR-Tools), y (3) *Fine-Tuning* LoRA supervisado sobre trazas válidas de resolución.

Estructura del Repositorio: Código reproducible en GitHub:

- `/data`: Esquemas JSON de personal y demanda de turnos.
- `/src`: `verifier.py` (*Ground Truth*) y `baseline_direct.py`.
- `/notebooks`: `Colab_Feasibility.ipynb` con perfilado de VRAM.